

SECURE VIBE CODING

How to Build Websites With AI — Without Leaving Security Behind

AI coding tools can generate authentication, APIs, forms and database logic in seconds. But **fast development** ≠ **secure development** — you still need to verify the generated code actually protects your application. You don't need to be a security expert first. Start with these **5 security layers**.

01 Always Validate on the Server

Relying only on client-side validation is a top beginner mistake. Attackers can send requests directly to your API, bypassing frontend checks entirely.

Your server should validate:

- Data types & required fields
- String length, allowed values, email formats
- File types/sizes, numeric ranges
- Authorization requirements

Rule: Client-side validation = UX. Server-side validation = the actual security boundary. Use both.

02 Protect Login With Rate Limiting

Without rate limiting, an attacker can send thousands of login attempts for credential stuffing or password guessing.

Base limits on:

- IP address
- Account identifier
- Device/session signals
- Time window

Also layer in: CAPTCHA/bot detection, MFA, risk signals, delays, credential-stuffing detection. Goal: make automated attacks expensive, not lock out real users.

03 Never Store Passwords in Plain Text

Passwords should not simply be “encrypted” — use a purpose-built password-hashing algorithm. Store the hash, never the original password. Salts make large-scale cracking far harder after a leak.

Flow: Password entered → hashing/verification → compare with stored hash → allow or reject. Prefer **Argon2id** for new apps; **bcrypt** is a solid alternative. Never invent your own algorithm.

04 Don't Leak Information Through Errors

“Email doesn't exist” lets attackers enumerate valid accounts. Use “Invalid email or password” instead — it reveals nothing about which part was wrong.

Also avoid exposing:

- Database errors & stack traces
- API keys & internal file paths
- Framework/debug information
- Sensitive user information

Dev vs Prod: Detailed errors are fine in development. In production, show safe messages; keep diagnostics in protected server logs.

05 Don't Build Authentication From Scratch If You Don't Have To

Login → verify → create session looks simple, but secure auth involves many moving parts where small mistakes become serious vulnerabilities.

Involves:

- Password hashing, session management, cookies
- CSRF protection, MFA, password reset
- Email verification, account recovery
- Session expiration, token handling, OAuth/OIDC, brute-force protection

Consider: Firebase Authentication, Clerk, Auth0, or an established library for your stack. Using a provider doesn't make you secure automatically — you still must configure authorization, APIs, DB access, secrets and sessions correctly.

BONUS — SECURE YOUR AI-GENERATED CODE

Don't ask AI to "build a secure website" and assume the result is secure. Make security part of the workflow. Before deploying, ask your coding assistant to review:

Authentication	Passwords securely hashed? Sessions/tokens handled safely? MFA available where appropriate? Password reset flows protected?
Authorization	Can User A access User B's data? Are permissions checked server-side? Are admin endpoints protected?
Input handling	Is every untrusted input validated server-side? Are file uploads restricted? Are dangerous inputs handled safely?
API security	Are sensitive endpoints authenticated? Are APIs rate-limited? Are excessive requests prevented? Are sensitive responses minimized?
Secrets	Search for API keys, DB credentials, private tokens, cloud credentials, secret env vars. Never hard-code secrets into frontend code or commit them to GitHub.

THE 30-MINUTE SECURITY REVIEW

STEP 1	STEP 2	STEP 3
Dependency checks Run dependency/security checks for your framework and packages.	Inspect auth Manually inspect authentication and authorization logic.	Test endpoints Test every API endpoint with unauthorized requests. Ask: "What happens if I remove the authentication?"
STEP 4	STEP 5	STEP 6
Ownership check Test if User A can access User B's resources, e.g. /api/orders/123 → /api/orders/124. Does the server verify ownership?	Prod config Debug mode off, HTTPS on, secure cookie attributes, no exposed secrets, CORS configured, security headers set.	Automated + expert review Run automated security testing and review results. For important apps, get a qualified security professional.

THE SECURE VIBE-CODING WORKFLOW

Prompt → AI generates app → Review architecture → Validate inputs (server) → Secure authentication → Implement authorization → Protect APIs → Secure secrets/config → Dependency checks → Test abuse cases → Deploy → Monitor & patch

Not: Prompt → AI generates website → Deploy. That skips every security checkpoint above.

FINAL TAKEAWAY

AI isn't automatically the security problem. The bigger risk is shipping AI-generated code without understanding or testing the security assumptions behind it. Use AI for speed. Use established security patterns for protection.

Authentication ≠ Authorization. A user being logged in doesn't mean they're allowed to access every resource.

AI-assisted development + secure engineering practices = websites actually ready to be trusted by users and clients.